Report

- Assignment 2 -

Artificial Intelligence Dept.

2277025 Eunsang Lee

Problem definition

Classifying images as smoking or not-smoking

AutoML tool used in this project

For this project, AutoKeras is used to find appropriate CNN model of this problem.

AutoKeras is built on top of TensorFlow (Keras) and is designed to automate the development of deep learning models for a variety of tasks. With minimal code, it can automatically design and train CNN architectures suited to the given problem.

Since I used a CNN architecture in Assignment 1, I thought it would be meaningful to compare my model with other CNN-based architectures. AutoKeras was best tool for my demand because I made my model using TensorFlow and It provides various CNN architectures based on the TensorFlow.

And AutoML uses proper data scaling and augmentation if it needs. So, I did data preprocessing only scaling the dividing the pixel values with 255 in order to each value to be [0, 1].

Development environment

OS : Google Colab

Python : 3.11.12 (default version in Google Colab)

Libraries

- kagglehub : 0.3.12
- tensorflow : 2.18.0
- sklearn : 1.6.1
- matplotlib : 3.10.0
- autokeras : 2.0.0

for a detailed list of required packages and their versions, please refer to the 'requirements.txt' file provided.

Results & interpretation

AutoKeras searches for several models and selects the optimal one based on a single metric value calculated on the validation set. Therefore, the performance of each model with the best hyperparameters selected by AutoKeras is compared based on the binary cross-entropy loss on the validation set. A detailed comparison is then made between the optimal model and the model I developed in Assignment 1.

The models discovered through AutoKeras include a vanilla CNN, ResNet, and EfficientNet.

The architecture and hyperparameters for each model are as follows:

1. CNN
   Block type : Vanilla CNN block
   Normalization : Used
   Augmentation : No
   Kernel size : 3
   Number of blocks : 1
   Number of layers : 2
   Number of filters : 32 for first layer, 64 for second layer.
   Max pooling : Used
   Depthwise separable convolution : No
   Dropout proportion: 0.25
   Classification Head : using flatten and dropout proportion 0.5
   Optimizer : Adam
   Learning rate : 0.001

2. ResNet
   Block type: ResNet block
   Normalization: Used
   Augmentation: Yes
   - Horizontal flip: Yes
   - Vertical flip: Yes
   - Contrast factor: 0.0
   - Rotation factor: 0.0
   - Translation factor: 0.1
   - Zoom factor: 0.0

Pretrained weights: Not used
ResNet version: ResNet50
Input resizing for ImageNet: Used
Classification Head: Using global average pooling
Dropout proportion: 0
Optimizer: Adam
Learning rate: 0.001

3. EfficientNet
   Block type: EfficientNet block
   Normalization: Used
   Augmentation: Yes
   - Horizontal flip: Yes
   - Vertical flip: No
   - Contrast factor: 0.0
   - Rotation factor: 0.0
   - Translation factor: 0.1
   - Zoom factor: 0.0
   Pretrained weights: Used
   EfficientNet version: B7
   Trainable: Yes (EfficientNet weights are fine-tuned)
   Input resizing for ImageNet: Used
   Classification Head: Using global average pooling
   Dropout proportion: 0
   Optimizer: Adam
   Learning rate: 2e-5

And the validation losses of each model are as follows:

- CNN in Aissignment1 : 0.3682
- Vanilla CNN : 0.5291047692298889
- ResNet : 0.626530647277832
- EfficientNet : 0.11375519633293152

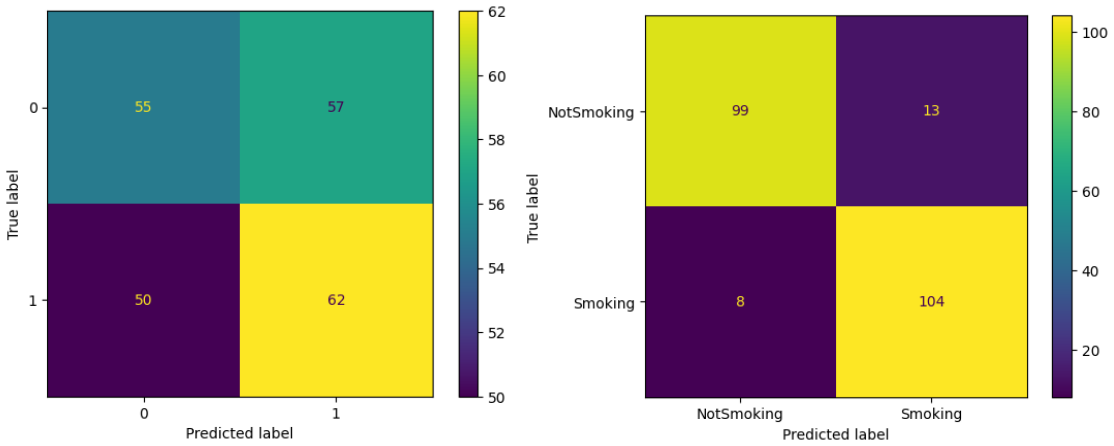
Interestingly, my model, which is also a vanilla CNN, outperformed the AutoKeras-generated vanilla CNN. Additionally, ResNet, which was expected to outperform the vanilla model, actually yielded worse performance. Among all models, EfficientNet

showed overwhelmingly superior performance. This may be due to EfficientNet's more complex architecture combined with the use of pre-trained weights and fine-tuning, which likely enabled it to learn more effectively from the dataset.

However, I still do not fully understand why ResNet, which also has a complex structure and could have used pre-trained weights, did not employ fine-tuning and ended up being the worst-performing model. It's puzzling that this approach was selected by AutoKeras.

I believe the reason why my vanilla CNN outperformed AutoKeras's version is related to memory constraints. Since AutoKeras needs to install and run multiple models, the available memory in Google Colab becomes more limited. This likely led AutoKeras to choose smaller model architectures in this constrained environment. In fact, the number of convolutional layers in my model is less than that of AutoKeras, suggesting insufficient data exploration due to limited model capacity.

Now, let's compare the test set performance between the model from Assignment 1 and the best model selected by AutoKeras, EfficientNet.



Left is confusion matrix of CNN in Assignment1(0 is NotSmoking and 1 is Smoking) and right is confusion matrix of EfficientNet.

Performance for the NotSmoking class

	accuracy	precision	recall	F1 score
Vanilla CNN in Assignment1	0.5223	0.5210	0.5536	0.5368
EfficientNet	0.91	0.93	0.88	0.90

The performance difference between the two models is clearly visible. My model produced a similar number of correct and incorrect predictions, while EfficientNet correctly predicted nearly all data points and achieved high precision and recall.

I believe this significant difference in test performance stems from EfficientNet's complex architecture and the use of pre-trained weights, which enabled the model to understand the dataset far more thoroughly.

Discussion on pros/cons of AutoML

One of the advantages of AutoML is that it reduces the burden of finding the most appropriate model and its hyperparameters within the given memory resources.

However, there are also some drawbacks. First, installing tools for AutoML consumes memory, which can prevent the development of larger models. This is evident from the fact that, although both are vanilla CNNs, the model I developed in Assignment 1 outperformed the one generated by AutoKeras. Additionally, there is always the possibility that a better combination of hyperparameters exists. The fact that ResNet showed worse performance than the vanilla CNN led me to consider this possibility.

Therefore, while AutoML is certainly a convenient tool, I believe we should not assume that it is always accurate or provides the absolute best solution.

Link for the remote source repository

You can see the code about assignment1 in here :
https://github.com/twoSilverUp/MLOps/tree/Assignment1

And you can see the code about using AutoKeras in here :
https://github.com/twoSilverUp/MLOps/tree/Assignment2